Multi-Tenancy & Processing Queues — Full Change Report

Date: 2026-10-02 / 03 · **Scope:** convert FDDT from single-tenant to multi-tenant, and split the document pipeline into three fair, rate-limited queues. **Round 2 (2026-10-03, production hardening)** is at the top; the original round follows from §0.

This file records **everything that was done, every decision taken (and why), every bottleneck or problem found, and what to change before production**. Items marked

> ⚠️ **PRODUCTION ACTION** — must be reviewed/changed before going live 🔶 **DECISION FOR YOU** — I chose a default; you may want it different 🐞 **PROBLEM FOUND** — a bug or limit hit during the work, and how it was handled

Detailed design docs: `multi-tenancy.md`, `processing-queues.md`.

R6. Browser test of the reviewer workflow (2026-10-03)

Chrome, company *QA Alpha Trading*, three cases processed end-to-end on real Azure (one English invoice, one school invoice, one Arabic hotel invoice). GIF of the escalate → L2 approve flow: `reviewer_escalate_and_l2_approve.gif` (browser downloads).

Step	Result
Submitter tries approve / reject / escalate (API)	✅ 403 each; risk tier hidden (`null`)
L1 approves a **high**-risk case	✅ dialog lists score + every triggered reason; button disabled until ≥ 10 characters (2/10 → disabled); approved; history "Approved · by Alpha Reviewer L1 · Reviewer L1" with the justification
L1 rejects	✅ "Reject case" disabled without a reason; API with empty reason → 422 "A reason is required..."; rejected with reason
L1 escalates	✅ dialog explains the one-way hand-off; status stays `pending_manual_review`, tier `l1 → l2`; "Escalated · L2" badge; L1 queue moves it to "Escalated to L2 · view only"
L1 on the escalated case (API)	✅ approve / reject / escalate → 403
Acting on decided cases	✅ 409 ("Cannot reject a case that is 'approved'", "This case is already rejected")
L2	✅ "Escalated queue (L2)" tab, escalated case sorted first; Escalate disabled; medium risk also needs a justification; approved; history shows both the L1 escalation and the L2 approval with roles
Audit log	✅ `case_approved` / `case_rejected` / `case_escalated` with `actor_role`, escalation `l1 → l2`
Submitter afterwards	✅ My cases: 2 Cleared, 1 Action Required; rejection reason visible
🐞 **Submitter's case timeline leaked the risk score**	✅ Fixed — `GET /cases/{id}/audit-log` returned `risk_assessment_completed` (tier, score, rules fired) and check results to the submitter. Now omitted/stripped for non-reviewers; test `test_submitter_timeline_hides_risk_score_and_check_results`

Step	Result
🐞 Escalated panel said "only an L2 reviewer **or admin**"	✅ Fixed — "only a Reviewer L2" (platform admins can't act)
🔶 Submitter still sees per-document check cards ("Issuer verification: Flagged" with match details)	**Open decision** — the case-detail API returns each document's checks to the submitter. Hiding them would fully blind submitters to which signals fired, but also hides useful feedback (e.g. a missing signature). Not changed
⚠️ Dev server only: case page flickers "Loading…" and remounts every ~1 s for a while after load	Not in the production build (verified with `vite preview`: stable, one fetch). Tied to Vite's dev server + the pdf.js module worker (respawned each cycle); my code-splitting change is **not** the cause (reproduced with eager imports). Use the production build when demoing
Minor	"My cases" shows a made-up subtitle `CASE-XXXX.pdf` under each case id instead of a real filename

R5. Browser test of PDF uploads and multi-tenancy (2026-10-03)

Run in Chrome against the local stack (real Azure for OCR/LLM/Blob), with two fresh companies (*QA Alpha Trading*, *QA Beta Schools*) and four users — accounts in `sample-documents/upload-tests/TEST_ACCOUNTS.md`, test files from `sample-documents/upload-tests/make_files.py`.

Area	Result
Upload — refused in the browser before sending	✅ `photo.jpg` ("only PDF files are accepted"), `empty.pdf` ("the file is empty")
Upload — refused by the server, specific message per file	✅ JPEG renamed `.pdf` ("This file is a JPEG image…"), text renamed `.pdf`, password-protected, truncated; 12 MB → 413 with both sizes; empty → 400
Upload — valid PDFs	✅ stored and processed end-to-end (score 87/high); the case holds exactly 2 documents, billing counts exactly their bytes
Isolation	✅ Beta L2 → Alpha case/timeline/file URL/signatures/reports/approve/reject/escalate/upload: all **404**; `company_id=<Alpha>` → 404; Beta sees 0 cases and no Alpha audit rows; UI shows "Case not found"
New company defaults	✅ empty issuer registry, 38 template rules
Roles	✅ L1: Settings hidden, page shows "Reviewer L2 access required", API 403; L2: own settings, user management 403; submitter: no Audit/Settings/Platform
Platform admin	✅ amber read-only Support view, no actions; every view logged (12 rows for one case visit); company users see none of them (UI and API)
Duplicate detection fix (R4)	✅ "Duplicate check complete" for both documents
🐞 Validation errors showed "Unprocessable Content"	✅ Fixed — `parseErrorDetail` now renders FastAPI 422 lists (e.g. "email: value is not a valid email address…")
🐞 "user created" rows in a company's log showed actor "System pipeline"	✅ Fixed — an actor the company session can't see (a platform admin) is now "Platform team"; test `test_company_log_names_platform_actions_platform_team`. Docs corrected: user_created/updated live in the user's company log
PDF viewer from `http://127.0.0.1:5173`	⚠️ "Couldn't load this file" — environment, not code: the storage account's CORS rule allows `http://localhost:5173` (works there). Add `127.0.0.1` to the rule if you use that origin

R4. Uploads, a permission bug, documentation and the production plan (2026-10-03)

Item	Status	What changed	Proof
Upload validation	✅ Done	Synchronous checks in the upload endpoint before anything is stored or queued: empty, > 10 MB, not a PDF by header bytes, extension mismatch, password-protected, corrupted (PyMuPDF; a repaired file without `%%EOF` counts as truncated). Distinct `{code, message}` per reason; frontend shows the message and refuses non-PDF / empty / > 10 MB before sending	`tests/test_upload_validation.py` ; all 38 sample PDFs (incl. 12 tampered) still pass
PDF only	✅ Done	Images (JPEG/PNG/TIFF) are no longer accepted — every forensic check is PDF-based. The message names the detected type ("This file is a JPEG image…")	same tests
🔶 Password policy	Decision	**Reject and ask** the uploader to remove the password; the server never handles document passwords. Owner-password-only PDFs (open without a password) are accepted	`test_password_protected_pdf_…` , `test_owner_password_only_pdf_is_accepted`
nginx body cap	✅ Done	`frontend/nginx.conf.template` `client_max_body_size` 50m → **11m**, so an oversized upload is refused at the proxy	—
🐞 **Duplicate detection failed in Postgres since round 2**	✅ Fixed	`store_page_hashes` used `ON CONFLICT DO UPDATE` , which needs UPDATE privilege; `fddt_app` has only SELECT + INSERT on `document_page_hashes` , so **every** duplicate check failed with "permission denied" (dev DB: 299 failed, last success 2026-10-02 18:35). The SQLite unit tests can't see grants. Now `ON CONFLICT DO NOTHING` (a page hash is a pure function of the immutable file) — no grant change, table stays insert-only	New `test_rls_postgres.py::test_every_upsert_the_pipeline_uses_works_under_the_app_role` runs every upsert twice as `fddt_app` ; it fails with the production error against the old code (negative control)
🐞 UI said platform access is "recorded in that company's audit log"	✅ Fixed	Three strings (company picker, Audit History, case Support view) now say the platform audit log, which the company does not see	build passes
Documentation	✅ Done	Every doc re-checked against the code: architecture, data flow, pipeline 01–11, API (33 paths / 41 operations), frontend, report, local setup, environment variables, Azure notes, user manual, READMEs. Database docs regenerated from the live DB by the new `docs/tools/gen_db_docs.py` (19 tables, RLS, grants). OpenAPI snapshot regenerated. PDFs rebuilt, including the multi-tenancy, queues, production-deployment and this report	—
Production plan	✅ Done	`deployment/production-deployment.md` : Container Apps architecture, S/M/L tiers, measured per-document AI cost, monthly costs from Azure list prices (UAE North), CLI runbook, monitoring, DR, go-live checklist	—

⚠️ **Re-run the failed duplicate checks** on the dev database if those cases matter (re-upload, or re-queue `run_duplicate_check` for the affected documents) — they will not re-run on their own.

R3. Follow-up fixes (2026-10-03)

Three of the four "should fix soon" items from R2.3 are done. The fourth (splitting classification out of `process_document`) is **deliberately not done** in this pass and is still open (R2.3).

Item	Status	What changed	Proof
Rate-limiter caps 85 % → 80 %	✅ Done	`backend/.env`: `AZURE_OPENAI_MAX_REQUESTS_PER_MINUTE=80`, `AZURE_OPENAI_MAX_TOKENS_PER_MINUTE=80000` (quota 100 / 100,000). `.env.example` and `config.py` comments now say ~80 %. Code defaults are unchanged (60 RPM / TPM off) because they don't assume a quota.	`tests/test_rate_limiter.py` passes. ⚠️ **Restart the vision and extraction workers** to pick it up. Expected cost: ~6 % less peak Azure OpenAI throughput (≈ 3.4 instead of ≈ 3.6 one-page docs/min).
Frontend code-splitting	✅ Done	`src/App.tsx`: every page except Login is `React.lazy` with one `Suspense` ("Loading…") around the routes.	`npm run build` (tsc + vite) and `oxlint` are clean. Main bundle **1.39 MB → 359 kB** (gzip 111 kB). Pages are now 4–106 kB chunks; shared Nav chunk 110 kB. The one chunk still > 500 kB is pdf.js's prebuilt `pdf.worker.min` (631 kB). It is a separate file that is only fetched when a PDF is shown, and it can't be split further. ⚠️ Not click-tested in a browser yet.
Queue metrics → monitoring + alerting	✅ App side done	`log_queue_metrics` (every minute) now logs `queue_metrics {json}` per queue, plus a **WARNING** `queue_alert {json}` line when the oldest waiting task > `QUEUE_ALERT_OLDEST_WAITING_SECONDS` (new setting, default 300, 0 = off). Format in `processing-queues.md` › Monitoring.	New `tests/test_queue_metrics_log.py` (2 tests): JSON per queue, alert only past the threshold, 0 disables. ⚠️ **Still a deployment step:** ship worker logs to Azure Monitor / Log Analytics and create an alert rule on `queue_alert`. That is not built, because CLAUDE.md says no cloud config yet.

Files: `backend/.env`, `backend/.env.example`, `backend/app/core/config.py`, `backend/app/tasks/usage_tasks.py`, `backend/tests/test_queue_metrics_log.py`, `frontend/src/App.tsx`, `docs/processing-queues.md`, this report.

R2. Round 2 — production hardening (2026-10-03)

Safety first: code backup `scratchpad/pre-round2-backup.tgz`, DB dump `scratchpad/docauth-pre-round2.dump`; the migration was run on a restored copy before the real dev DB. New migration: `c9e3a1b5d7f4`.

R2.0 Status of the eight requested fixes

#	Fix	Status	Proof
1	No BYPASSRLS — policy-based platform access	✅ Done	No role has BYPASSRLS; 18 `platform_all` policies; **whole migration chain run as a NOSUPERUSER/CREATEROLE owner** on a fresh DB; RLS suite 15/15 on both that DB and the real one
2	Forensics on Linux, prefork, horizontally scaled	✅ Done	Containers run `--pool=prefork`; `--scale` verified (2 replicas); at the real quota forensics drains first (35 s vs vision 315 s)
3	Platform-admin audit rows platform-only, no dedup	✅ Done	Rows stored with `company_id NULL` (RLS hides them from every company); company views (Audit History, case timeline, report PDF) exclude them; dedup default 0; tests (re-verified 2026-10-03, see §5)

#	Fix	Status	Proof
4	Fair-share per company	✅ Done	Q-2 now passes: B waited 0.0–0.2 s (forensics) / 1.2 s (vision) vs A's median 30 s / 44 s
5	Global risk-rule templates seeding new companies	✅ Done	`risk_rule_templates` = Default Company's 38 rules; new company populated immediately; template edits don't touch existing companies; Platform › Rule templates screen
6	Idempotent check writes	✅ Done	Upserts + unique constraints; simulated worker crash and redelivery leave exactly one row
7	Connection pooling + PgBouncer (transaction mode)	✅ Done	PgBouncer service; NullPool behind it; leak test incl. negative control passes; arithmetic documented
8	Calibrate image-token estimate on real usage	✅ Done	96 real calls measured; estimator replaced; real-Azure burst 40 × 200, **0 × 429**

Tests: **452 passed** (twice, full suite including Postgres RLS, PgBouncer and live-Redis tests; nothing skipped).

R2.1 Decisions I made (🔶 change if you disagree)

- 🔶 **Platform-access rows are stored with** `company_id = NULL` (target company in `event_data`), not just filtered out of company views. That makes them invisible to company users at the **database** level (RLS), not only in the UI. Existing rows (0 on the dev DB) were moved by the migration — no row deleted, no content changed.
- 🔶 **Platform admins see platform-access rows in both** the platform-level log and a company's log view.
- 🔶 **The template = the current version of each of Default Company's rules: 38 rules.** (The "42" in the brief were 38 rules + 4 older versions of some of them.) Inactive template rules are not copied to new companies. Templates are plain editable rows (no version history of their own); every edit is in the platform audit log with old → new values.
- 🔶 **Fair-share bucket = 5** (`FAIR_SHARE_BUCKET_SIZE`): a company's tasks drop one priority level per 5 outstanding tasks on a queue, 10 levels. Priority is computed when a task is published (from live counters), not by re-sorting already-queued tasks — simpler and robust; the load tests show it meets Q-2.
- 🔶 **Duplicates collapsed by the migration** (from earlier retries): 6 check rows, 1 page hash, 1 assessment on the dev DB — newest check/hash kept; for assessments the one a report references (else the earliest) kept.
- 🔶 `max_tokens` **lowered** for the four fixed-shape LLM calls (entity match 4000→512, visual review 1500→640, signature detection 800→480, comparison 800→320) based on measured outputs (max 155 / 204 / 144 / 47). Classification stays 4,000 (its output grows with the document). +26 % throughput.
- 🔶 **Behind PgBouncer, processes keep no pool of their own** (NullPool). Without PgBouncer the default per-role pool stays 5 + 10, because a 16-thread worker would starve on a smaller one.

R2.2 Problems found during this round (🐞) and how they were fixed

🐞 Problem	Fix
The old token estimate UNDER-reserved the two most common calls (visual review 0.54×, classification 0.64× of real) — the limiter would have allowed more than the real quota at full load	Patch-based image estimate + measured chars/token (1.93, Arabic-dense) + 5 % margin; every call now ≥ 1.05× real
Flat per-image cost is wrong: full pages vs. signature crops differ ~5×	Image tokens from pixel size (32-px patches × 1.62)

🐞 Problem	Fix
Tasks held a DB transaction across Azure calls/downloads → in PgBouncer transaction mode each pinned a server connection; 32 threads exhausted the pool and **stalled the API** (uploads 93 s for 100, monitor timeouts)	Tasks commit before every slow step; task sessions keep attributes after commit; upload endpoint releases its transaction before the blob upload. After the fix: uploads 50 s, drain 92 s (was 120 s), no timeouts
PostgreSQL 16 rejects `ALTER ROLE … NOSUPERUSER/NOBYPASSRLS` from a non-superuser (found by the non-superuser migration test)	Attributes set on `CREATE ROLE` only; `ALTER` sets the password; migration asserts the roles are unprivileged
`nproc` **ignores container CPU limits** → 2 forensics replicas each started 4 processes on a 4-core host	`start-worker.sh` reads the cgroup quota (`cpu.max`); compose sets `cpus:` per replica
`platform_admin_access` rows were visible to company reviewers (by design in round 1)	Now platform-only (above)
Celery's Redis priorities split each queue into 10 lists — the queue monitor would have under-counted	Monitor sums all priority lists; oldest-waiting checks each

R2.3 ⚠️ What to know / do before production (new in this round)

- ⚠️ **Real throughput ceiling at the current quota is ≈ 3.6 one-page documents/minute (~210/hour)** — *lower* than round 1's "5–6/min", which rested on the under-estimate. Raise the Azure OpenAI TPM (e.g. to 500k–1M) for more. The calibrated limiter will keep using exactly what you configure.
- ⚠️ `process_document` **couples Document Intelligence with the Azure OpenAI classification call.** When the token limiter saturates, extraction threads wait on the OpenAI quota and a new document's OCR waits for a free thread (B waited 14–50 s on extraction while A barely did). Recommended next change: split classification into its own `vision_queue` task (persist OCR word boxes between them). **Still open** (not done in R3).
- ⚠️ **One simulated 429 in 123 calls** at full token-limiter saturation in the real-quota load test (a race between two independent sliding windows). The real SDK retries it; the real-Azure burst had none. For extra margin set the caps to **80 %** of quota instead of 85 %. ✅ **Done in R3.**
- ⚠️ **Run workers and the API on separate nodes.** On one 4-core box prefork forensics takes the CPU the API needs (by design it uses all of its allowance) — cap `FORENSICS_WORKER_CPUS` if they must share.
- ⚠️ **PgBouncer** `userlist.txt` **holds dev passwords** — use SCRAM secrets from a vault in production; keep Postgres `max_connections` ≥ 70 (default 100 is fine) with the shipped PgBouncer sizing.
- ⚠️ **Re-run** `python -m scripts.calibrate_llm_tokens` whenever the model, prompts or page render size change (costs a few cents).
- ⚠️ The **migration owner needs CREATEROLE** (no superuser). On Azure Flexible Server the admin user has it.

R2.4 New / changed files

Backend: `app/tasks/fairshare.py` , `app/services/check_store.py` , `app/models/risk_rule_template.py` , migration `c9e3a1b5d7f4_production_hardening.py` , `scripts/calibrate_llm_tokens.py` , `scripts/azure_limiter_burst.py` , tests `test_pgbouncer_rls.py` , `test_idempotent_tasks.py` , `test_rule_templates.py` , `test_fairshare.py` ; changed: migration `a7c1e9d3f5b2` (no BYPASSRLS, CREATE ROLE attributes), `tenant_access.py` , `audit.py` , `cases.py` , `platform.py` , `db/session.py` , `core/config.py` , `celery_app.py` , `usage_tasks.py` , `queue_monitor.py` , `llm_service.py` , `issuer_service.py` , `risk_rule_seed.py` , `risk_scoring_service.py` , `case_report_data.py` , `duplicate_check.py` , every task module (`open_task_session` , commit before slow steps, upserts), models (unique constraints), `start-worker.sh` , `loadtest/*` , `.env.example` . Frontend: `pages/PlatformRuwhen

Round-1 #	Status now
1 Azure OpenAI quota	Measured & calibrated; real ceiling ≈ 3.6 docs/min (see R2.3); still needs a TPM increase

Round-1 #	Status now
2 FIFO fairness	✅ Fair-share built and verified
3 Forensics bottleneck	✅ Linux prefork + scaling; fastest queue at the real quota
4 BYPASSRLS on Azure	✅ Removed everywhere
5 Dev DB passwords	⚠️ Still to set in production (now also PgBouncer `userlist.txt`)
6 Audit dedup	✅ 0 (every access logged), rows platform-only
7 Limiter fails open	🔶 Unchanged — no fail-open event occurred in any test
8 User manual / browser walkthrough	⚠️ Still open

0. Summary of the most important findings (round 1)

#	Finding	Severity	What to do
1	**Azure OpenAI quota caps the whole platform at ~5–6 documents/minute** (gpt-4.1-mini: 100 RPM, **100,000 TPM**). Tokens, not requests, are the limit.	⚠️ High	Raise TPM on the Azure deployment and/or lower `max_tokens` on LLM calls (see §6.1)
2	**Strict FIFO cannot let a small upload jump an existing big backlog** (Load test 2: Company B waited behind all 100 of A's docs).	🔶 Decision	Approve "fair-share per company" scheduling if wanted (see §6.2)
3	**Forensics queue is the throughput bottleneck** on the local 4-core Windows box.	⚠️ Medium	Run forensics on Linux with prefork, scale containers (see §6.3)
4	**Managed Azure PostgreSQL may not allow `BYPASSRLS`**, which the platform DB role uses.	⚠️ High (for Azure deploy)	Use the policy-based alternative in §6.5
5	Default DB role passwords (`fddt_app_local`, `fddt_platform_local`) are dev-only.	⚠️ High	Set long random `DATABASE_APP_PASSWORD` / `DATABASE_PLATFORM_PASSWORD`
6	Platform-admin audit rows are **de-duplicated for 5 minutes** for identical reads.	🔶 Decision	Set `PLATFORM_ACCESS_AUDIT_DEDUP_SECONDS=0` if every request must be logged
7	The rate limiter **fails open** if Redis is down (calls go through un-throttled).	🔶 Decision	Keep (pipeline keeps running) or change to fail-closed (§6.7)
8	`docs/user-manual.md` and some other docs still describe the old `admin` role. Frontend not click-tested in a browser.	⚠️ Medium	Update manual; do a UI walkthrough

1. Safety steps taken before changing anything

- The project is **not a git repository**, so a full code backup was made first: `scratchpad/pre-multitenant-backup.tgz` (backend, frontend, docs, compose, CLAUDE.md, README).
- The real dev database was dumped before migration: `scratchpad/docauth-pre-multitenant.dump` (`pg_restore` to roll back).

- The migration was first run on a **restored copy** (`docauth_mtest`), including a downgrade → upgrade round trip, and the RLS tests were run there before touching the real DB. The copy was dropped afterwards.

> ⚠️ **PRODUCTION ACTION:** put the project under git before further work, and take a DB backup before running `alembic upgrade head` on any shared/production database.

---

2. Role model (replaces the old one)

Role	Company	Permissions
`user`	exactly one	submit cases/documents, track own cases
`reviewer_l1`	exactly one	review company queue, approve/reject/escalate (L1)
`reviewer_l2`	exactly one	L1 + escalated cases + **own company's** Issuer Registry & Risk Rules
`platform_admin`	**none**	create companies, create **all** users (company + role), billing dashboard, queue monitor, read-only audited support access to any company

Decisions:

- 🔶 `platform_admin` **is NOT on the rank ladder.** It never inherits reviewer powers: it **cannot approve, reject, escalate, upload, create cases or generate reports**. Spec said "full read access for support" — I interpreted that as read-only. (If support needs to act on cases, add it explicitly.)
- 🔶 **Platform admins CAN edit a company's Issuer Registry and Risk Rules** (with `company_id`), and every such change is audited in that company's log. Reason: support/onboarding needs it. Remove if not wanted.
- **No company-level user management** at all — `Settings › Users` is platform-admin only (as specified).
- The old `admin` role was migrated to `platform_admin`.
- Moving a user to another company, or changing them across the platform/company boundary, **invalidates their existing JWT** (they must log in again). Plain role changes inside a company take effect on the next request (role is always re-read from the DB).
- Suspending a company (`is_active=false`) blocks login and existing tokens for all its users.

---

3. Database changes

Migration: `backend/app/db/migrations/versions/a7c1e9d3f5b2_multi_tenancy.py`

- New table **`companies`** (id, name unique, is_active, created_at, updated_at).
- New table **`company_usage_stats`** (company_id, usage_date, cases_created, documents_uploaded, files_stored, storage_bytes) — one row per company per UTC day.
- `company_id` **added to every tenant table:** users, cases, documents, document_checks, document_page_hashes, cross_document_findings, case_actions, case_risk_assessments, risk_scores, risk_rules, risk_settings, issuer_registry, signature_references, signature_matches, case_reports, audit_log.
  - NOT NULL everywhere except `users` (NULL = platform admin, enforced by CHECK constraint `(role='platform_admin') = (company_id IS NULL)`) and `audit_log` (NULL = platform-level event).

- 🔶 The spec listed `extracted_fields` and `duplicate_matches` tables — **they don't exist**. Extracted fields are a column on `documents`; duplicate hashes live in `document_page_hashes`. Both are covered by their table's `company_id`. I also added `company_id` to tables the spec didn't list but which hold company data (case_actions, risk_settings, cross_document_findings, risk_scores, document_page_hashes).
- Risk rules unique key changed to `(company_id, rule_id, version)`; `risk_settings` is one row per company.
- `user_role` enum gained `platform_admin` (the old `admin` value stays in the Postgres enum because Postgres can't drop enum values; nothing uses it).

Data migration (done on your real dev DB):

- One company **"Default Company"** created (`DEFAULT_COMPANY_NAME`); all 42 cases, 78 documents, 514 checks, 750 audit rows, 42 rules, 8 issuers, 14 reports etc. assigned to it.
- `admin@example.com` → `platform_admin`, no company. 4 reviewers and 2 users kept their roles.
- Verified **byte-for-byte**: every case's id, number, status, risk tier, document count, size and file hashes are identical before/after.
- Usage counters back-filled from existing data.

🐞 **`seed.py` resets the `admin@example.com` password to `ChangeMe123!`** every time it runs (it always did; `start-api.sh` runs it on every start).

> ⚠️ **PRODUCTION ACTION:** set `SEED_ADMIN_EMAIL` / `SEED_ADMIN_PASSWORD`, or remove `seed.py` from `start-api.sh` in production.

---

4. Tenant isolation — two layers

4.1 Application layer

Files: `app/db/tenancy.py`, `app/db/session.py`, `app/api/auth.py`, `app/api/tenant_access.py`, `app/api/case_access.py`

- JWT now carries `role`, `company_id`, `is_platform_admin`; the token is re-validated against the DB on every request.
- Every endpoint has **two mandatory checks**: role check + tenant check (request session bound to the caller's company by the `get_tenant_db` dependency — a route can't forget it).
- Every DB session is **unbound / company / platform**:
  - *unbound* → refuses any query or write on tenant tables (raises an error);
  - *company* → every ORM query on a tenant table gets `company_id = X` added automatically, inserts are stamped with X, a row of another company is refused at flush;
  - *platform* → explicit bypass path.
- Service functions also take `company_id` explicitly (`load_current_rules`, `get_risk_settings`, `match_issuer`, `run_duplicate_check`, `score_case`, `pipeline_status`, `latest_assessment`, `get_case_flags` …).
- 🔶 **Cross-company id → 404** (not 403), so ids can't be probed. Matches the existing submitter rule.
- Celery tasks now receive `(document_id, company_id)` and open a company-bound session. Old queued messages without company are resolved once via the platform session.

🐞 **PROBLEM FOUND — identity-map leak:** `session.get()` returns cached objects without running a query, so the automatic filter couldn't see it. If one session had loaded company B rows and was then re-bound to A, `get()` could return a B row. **Fixed:** binding to a company now evicts other companies' objects from the session cache (test:

`test_company_session_never_returns_another_companys_rows_even_without_a_filter`). Production sessions are bound once per request anyway, and RLS would still block it.

4.2 Database layer — PostgreSQL Row-Level Security

- Two new login roles created by the migration:
  - `fddt_app` — NOSUPERUSER, **NOBYPASSRLS** — used by every company request and every pipeline task.
  - `fddt_platform` — **BYPASSRLS** — used only for login/identity lookup, platform-admin screens, usage reconciliation.
  - The owner role (`DATABASE_URL`, `docauth`) is now used **only** by Alembic and `seed.py`.
- RLS enabled on all tenant tables + `companies` + `company_usage_stats`, policy `company_id = fddt_current_company_id()` (reads the transaction-local `app.current_company_id`). No company set ⇒ **zero rows**.
- The app sets the company at the start of **every transaction** (`set_config(..., is_local=true)`), so a pooled connection never leaks one request's company into the next.
- Least-privilege grants, which also make these tables **append-only in the database itself**: `audit_log`, `risk_rules`, `case_risk_assessments`, `case_reports` (SELECT + INSERT only). `users` is SELECT-only for `fddt_app`.
- 🔶 No default privileges: any **future tenant table is unreachable** by the app until its migration adds RLS + grants. (Forces developers to think about isolation; see `enable_tenant_rls()` in the migration.)

> ⚠️ **PRODUCTION ACTION:** set strong `DATABASE_APP_PASSWORD` and `DATABASE_PLATFORM_PASSWORD` (defaults `fddt_app_local` / `fddt_platform_local` are for local dev only).

> ⚠️ **PRODUCTION ACTION:** the app now opens **two connection pools** (app + platform). Size Postgres `max_connections` accordingly (each engine uses SQLAlchemy defaults: 5 + 10 overflow per process, times API workers + Celery worker processes).

---

5. Platform admin support access (audited, platform-only)

> **Updated in round 2 (R2.0 #3).** In round 1 these rows went into the company's own audit log and the company's reviewers could see them. That is **no longer the case**. Below is the current behaviour, verified on 2026-10-03.

- Opening any company's case / case list / audit log / settings writes a `platform_admin_access` row: "*Platform admin ops@x.com viewed Company B / Case CASE-1A2B3C4D*", with method and path.
- **Only platform admins can see these rows. No company user (User, Reviewer L1, Reviewer L2) can see them, in any view.** Company users have no way to tell that platform access happened.
  - **Database layer:** the row is written through the platform session with `company_id = NULL`. The target company goes in `event_data.company_id`. Row-Level Security hides NULL-company rows from the `fddt_app` role that every company request uses. So even a raw SQL query with no filter cannot return them (`app/api/tenant_access.py`).
  - **Application layer (defence in depth):** every company-facing read also excludes the event type explicitly. That covers Audit History (`app/api/audit.py` `_scope_filter`), its event-type filter list, the case timeline (`app/api/cases.py`, GET `/cases/{id}/audit-log`) and the per-case forensic report PDF (`app/services/case_report_data.py` `audit_rows`).
  - **Platform admins** see the rows in the platform-level log, and in a company's log view (filtered by `event_data.company_id`). Opening a company's log is itself audited.

- Platform admins never get a mixed cross-company case list. The queue takes one company at a time (company picker in the UI).
- **No de-duplication:** every access is logged (`PLATFORM_ACCESS_AUDIT_DEDUP_SECONDS=0` , the default). A value > 0 would collapse identical reads by the same admin within that window. *(Round 1 used 300 s.)*
- The platform-level audit log (company created, user created, usage reconciled, platform access) is a separate view (checkbox in Audit History for platform admins).
- Tests that prove it:
  - `tests/test_multi_tenancy.py::test_company_users_never_see_platform_admin_access` covers every company role: Audit History, the `event_type` filter, the event-type list and the case timeline.
  - `tests/test_rls_postgres.py::test_platform_admin_access_rows_are_invisible_to_the_app_role` checks raw SQL as `fddt_app` with no event-type filter.
  - `tests/test_multi_tenancy.py::test_platform_admin_access_is_read_only` checks the platform-admin side.
  - Re-run 2026-10-03: all three pass (the RLS one against the real Postgres).

---

6. Bottlenecks found and how to solve them

6.1 ⚠️ Azure OpenAI quota (gpt-4.1-mini) — the real throughput ceiling

- I read the deployment's real limits from the `x-ratelimit-*` headers (one ~10-token test call): **`x-ratelimit-limit-requests = 100` / min, `x-ratelimit-limit-tokens = 100,000` / min.**
- Azure charges each request **prompt tokens + the full `max_tokens` reservation** against TPM up front. Our calls reserve: classification **4,000**, visual review **1,500 (run twice per page)**, signature detection/comparison **800**, plus ~1,100 tokens per page image.
- **≈ 13–15k tokens per one-page document ⇒ ~5–6 documents/minute (~350/hour) for the whole platform**, no matter how many workers you add.
- Implemented: a **token-weighted global limiter** in addition to the request limiter, set in `backend/.env` to 85%: `AZURE_OPENAI_MAX_REQUESTS_PER_MINUTE=85` , `AZURE_OPENAI_MAX_TOKENS_PER_MINUTE=85000` . Without it, an RPM-only limiter would have caused 429 storms at load.

> ⚠️ **PRODUCTION ACTION (pick one or more):**
>
> 1. **Increase the deployment's TPM quota** in Azure (e.g. 500k–1M TPM) and update the two env vars to ~80% of the new values (R3).
> 2. **Lower `max_tokens`** where outputs are small: classification 4000 → ~1500, visual review 1500 → ~800 (check real response sizes first). This alone can roughly double throughput.
> 3. Consider a **Provisioned/PTU** deployment or a second deployment/region for vision calls.
> 4. The per-image estimate (`_IMAGE_TOKEN_ESTIMATE = 1100` in `llm_service.py`) is an approximation; tune it from real `usage` data.

6.2 🔶 Fairness: strict FIFO vs. "small upload jumps big backlog"

- Every document step is its own task, all companies share FIFO queues, no priority (as specified).
- **Load test 1 passed:** 27 concurrent uploads from 3 companies were processed interleaved (Kendall τ = 0.96 vs submission order).
- **Load test 2 did NOT meet the expectation:** Company B's single document, submitted right after A's 100-document upload, started after **all 100** of A's on every queue (waited 15 s extraction, 70 s vision, 144 s forensics). A FIFO queue cannot let a later job jump work that is already queued.

- **Solution (not built — needs your approval):** *fair-share per company* — order each new task by how much work its company already has waiting (e.g. Celery Redis priority computed from the company's queued count, or one queue per company drained round-robin). No company gets permanent priority; each gets an equal share. It does reorder the queue, which you said should be your decision.

6.3 ⚠️ Forensics queue is CPU-bound

- In the load test the forensics queue reached **649 waiting / oldest 264 s** while the Azure queues idled. Cause: 4 threads on a 4-core **Windows** machine (Windows can't use Celery's prefork pool; threads share the Python GIL).
- **Solution:** run forensics workers on **Linux** with `start-worker.sh forensics` (prefork, one process per core) and scale containers: `docker compose --profile workers up -d --scale forensics-worker=N`. This queue has no rate limit — more cores = more throughput.

6.4 Celery's built-in `rate_limit` is per worker, not global

- Celery's own `rate_limit` stops being a ceiling once you run 2+ worker containers.
- **Solved:** Redis-backed sliding-window limiter (`app/services/rate_limiter.py`) applied at **each Azure API call** (not per task), shared by all processes. On a 429 from Azure, **all** workers pause that service for the Retry-After time.
- Measured: Document Intelligence peaked at exactly **10 calls/s** (cap 10, Azure quota 15), **0 × 429**.

6.5 ⚠️ BYPASSRLS on managed Azure PostgreSQL

- The migration creates roles and grants `BYPASSRLS`; that needs superuser. Local docker Postgres allows it. **Azure Database for PostgreSQL Flexible Server may not let you grant BYPASSRLS.**
- **Solution if it's refused:** create `fddt_platform` without BYPASSRLS and add a policy that grants it all rows instead:

  ```
  CREATE POLICY platform_all ON <table> FOR ALL TO fddt_platform USING (true) WITH CHECK (true);
  ```

  (repeat for every tenant table). Same effect, no superuser needed. Create the two roles once with the Azure admin account before running the migration.

6.6 Monitoring accuracy

- Queue monitor (Platform › Processing queues, `/platform/queues`) tracked the load spike correctly; mean difference vs. direct Redis count 1.7 (samples taken moments apart while draining).
- "Consumers online" uses Celery remote control with a 1 s timeout — can show "unknown" under heavy load.
- A beat task logs one `queue_metrics` line per queue every minute.

> ⚠️ **PRODUCTION ACTION:** ship those log lines to Azure Monitor / Log Analytics and alert on `oldest_waiting_seconds` (e.g. > 300 s). *(R3: the app now logs JSON `queue_metrics` lines and a WARNING `queue_alert` line past `QUEUE_ALERT_OLDEST_WAITING_SECONDS`. The log shipping and the alert rule are still to set up at deployment.)*

6.7 🔶 Rate limiter fails open

- If Redis is unreachable, the limiter logs a warning and lets calls through (SDK retries are then the only protection), so the pipeline doesn't stop. If you'd rather stop than risk 429s, change `_client()` in `rate_limiter.py` to raise instead.

6.8 `task_acks_late` side effects

- Enabled for fair dispatch (worker holds only the task it runs). Side effect: if a worker crashes mid-task the task is **re-run**, which can write a duplicate check row (scoring always uses the latest row, so the score is correct). Redis `visibility_timeout` (default 1 h) must stay above the longest task.

7. Billing & usage dashboard

- Platform › Billing & usage: **one row per company** — cases created, documents uploaded, files stored (documents + reports), storage added in the period, total storage/files all-time; periods: this month, last month, last 30 days, this year, all time, custom range (UTC days).
- Fast counters in `company_usage_stats`, incremented with an atomic upsert **in the same transaction** as the upload/case/report (a failed upload never inflates them).
- **Nightly reconciliation** (Celery beat, 02:00 UTC, `USAGE_RECONCILIATION_HOUR_UTC`) recomputes from source tables and fixes drift; "Reconcile now" button does it on demand. Corrections are audited.
- Storage = exact bytes recorded at write time. 🔶 Signature crop images are **not counted** (tiny, derived).
- 🔶 Nothing deletes cases/documents today. **If deletion is added, it must decrement counters** (or the nightly job corrects it next day).
- Verified in load test: Loadtest A = 10 cases / 109 docs, B = 10 / 10, C = 9 / 9 — exactly what was uploaded.

> ⚠️ **PRODUCTION ACTION:** run exactly **one** `celery beat` process (or the nightly job runs twice).

8. Blob storage

- New layout: `companies/{company_id}/cases/{case_id}/documents|reports|signatures/...`
- `ensure_company_blob()` refuses to sign a URL under another company's prefix.
- 🔶 **Existing files were NOT moved** (originals are immutable). Old paths still work; their DB rows are company-scoped. If you want a uniform layout, a one-off copy job can be written.
- 🐞 Windows 260-char path limit hit by the **load-test stub** (deep nested path) — fixed in the stub (hashed file names). Not an issue for real Azure Blob (1024-char names).

9. Processing queues — configuration

Queue	Tasks	Limit	Workers env
`extraction_queue`	OCR (Doc Intelligence) + classification	10 calls/s global	`EXTRACTION_WORKER_CONCURRENCY` (4)
`vision_queue`	visual review, signature detection/comparison, issuer check	85 RPM + 85k TPM global (80 / 80k since R3)	`VISION_WORKER_CONCURRENCY` (4)
`forensics_queue` (+ legacy `celery`)	metadata, ELA, copy-move, duplicates, cross-doc, scoring	none	`FORENSICS_WORKER_CONCURRENCY` (0 = per core)

- Start: `start-worker.sh extraction|vision|forensics`, `start-beat.sh`; Docker: `--profile workers`.
- `worker_prefetch_multiplier=1`, `task_acks_late=True`.
- Forensics worker also drains the old `celery` queue so pre-upgrade messages aren't stranded.
- 🔶 No per-company/user priority (as specified).

10. Frontend changes

- Roles: `platform_admin` added, `admin` removed; `hasRank` excludes platform admin.

- **Company picker** (platform admins) on Case queue, Dashboard, Audit History, Issuer Registry, Risk Rules — choice remembered per browser; amber "support access — audited" notice.
- Case detail: read-only **"Support view"** banner for platform admins; no actions, no report export.
- Nav shows company name + role; "New upload" hidden for platform admins.
- Settings: Users tab platform-only, with Company column, company filter, move-user, company selector in Add user.
- New pages: **Platform › Companies** (create/suspend), **Billing & usage**, **Processing queues**.
- Production build and type-check pass.

> ⚠️ **PRODUCTION ACTION:** do a full click-through of every role in a browser (not done — only build/type-checked). Bundle is 1.39 MB (Vite warns > 500 kB) — add route-level code-splitting. *(✅ R3: main bundle now 359 kB.)*

---

11. Problems found during testing (and fixes)

🐞 Problem	Fix
`session.get()` could bypass the tenant filter via the identity map	Evict other companies' objects on bind (§4.1)
SQLite `CAST(.. AS DATE)` returned a number → reconciliation crash in tests	Use `date()` on SQLite; `timezone('UTC', ...)::date` on Postgres
Load-test upload 500s	Stub's Windows path length; switched to hashed filenames
Rate-limit test flaked by ~10 ms thread jitter	Limiter is correct by admission time; test window allows jitter
Old tests assumed one global `admin`	~30 test files updated; new tenant fixtures

---

12. Tests

- **430 passing**: 415 app-layer (incl. new `tests/test_multi_tenancy.py`), **11 PostgreSQL RLS** (`tests/test_rls_postgres.py`, raw SQL with **no** company filter, run against the real dev DB), **4 rate-limiter** tests against live Redis.
- RLS tests need `FDDT_RLS_TEST_DATABASE_URL`; limiter tests skip if Redis isn't reachable.
- Load test harness: `backend/loadtest/` (real API/workers/Postgres/Redis; only Azure + Blob stubbed).

Spec test	Result
MT-1 create companies & users	✅
MT-2 A's L2 sees A; B by id → 404	✅
MT-3 L2 can't reach Settings › Users	✅ (403)
MT-4 platform admin opens B case, audited	✅
MT-5 billing separated & matches	✅
MT-6 RLS alone blocks cross-tenant	✅
MT-7 pre-migration data intact	✅ (verified on real DB)

Spec test	Result
Q-1 interleaved processing	✅ τ = 0.96
Q-2 small upload near front	❌ by design of strict FIFO — see §6.2
Q-3 throttled, no 429 storms	✅ 0 × 429
Q-4 monitor reflects reality	✅

13. New / changed configuration (`backend/.env`)

```
DATABASE_APP_USER=fddt_app
DATABASE_APP_PASSWORD=...            # ⚠️ change
DATABASE_PLATFORM_USER=fddt_platform
DATABASE_PLATFORM_PASSWORD=...       # ⚠️ change
DEFAULT_COMPANY_NAME=Default Company
PLATFORM_ACCESS_AUDIT_DEDUP_SECONDS=0    # round 2: every access logged (was 300 in round 1)
AZURE_DOCUMENT_INTELLIGENCE_MAX_CALLS_PER_SECOND=10
AZURE_OPENAI_MAX_REQUESTS_PER_MINUTE=80     # measured quota 100 (85 until R3)
AZURE_OPENAI_MAX_TOKENS_PER_MINUTE=80000    # measured quota 100,000 (85000 until R3)
QUEUE_ALERT_OLDEST_WAITING_SECONDS=300      # R3: WARNING queue_alert log line; 0 = off
EXTRACTION_WORKER_CONCURRENCY=4
VISION_WORKER_CONCURRENCY=4
FORENSICS_WORKER_CONCURRENCY=0
USAGE_RECONCILIATION_HOUR_UTC=2
```

(The three quota lines were appended to your local `.env`; all are documented in `.env.example`.)

14. Production-readiness checklist

Must do before go-live

- [ ] Put code in git; back up DB before migrating.
- [ ] Strong passwords for `fddt_app` / `fddt_platform`; strong `JWT_SECRET_KEY`; secrets in Key Vault.
- [ ] On Azure Postgres: pre-create roles; use the policy alternative if BYPASSRLS is refused (§6.5).
- [ ] Raise Azure OpenAI TPM and/or lower `max_tokens` (§6.1); re-set the two env limits.
- [ ] Remove `seed.py` from startup or set `SEED_ADMIN_*` (it resets the admin password).
- [ ] Run forensics workers on Linux/prefork and scale them (§6.3); exactly one beat process.
- [ ] Alerts on queue `oldest_waiting_seconds` and on Azure 429 counters.
- [ ] Browser walkthrough of every role; update `docs/user-manual.md` (still says `admin`).
- [ ] Remove the load-test companies ("Loadtest A/B/C …") from the dev DB if it will be reused.

Your decisions

- [ ] Fair-share scheduling per company? (§6.2)
- [ ] Platform admin read-only on cases — OK? Platform admin editing company settings — OK?
- [x] Audit de-dup window 300 s or 0? → **0** (round 2); platform-access rows are platform-only (§5)
- [ ] Rate limiter fail-open or fail-closed?
- [ ] Move legacy blobs to the new company-prefixed layout?

Not built (out of scope / future)

- Deletion of cases/documents (and counter decrement), per-company priority tiers, password reset/MFA, company self-service user management (explicitly excluded), cross-company duplicate detection (deliberately excluded for privacy).

---

15. Files added / changed

Added (backend): `app/models/company.py`, `app/models/company_usage_stats.py`, `app/db/tenancy.py`, `app/api/tenant_access.py`, `app/api/platform.py`, `app/services/usage_service.py`, `app/services/rate_limiter.py`, `app/services/queue_monitor.py`, `app/tasks/tenant.py`, `app/tasks/usage_tasks.py`, migration `a7c1e9d3f5b2_multi_tenancy.py`, `start-beat.sh`, `loadtest/*`, `tests/test_multi_tenancy.py`, `tests/test_rls_postgres.py`, `tests/test_rate_limiter.py`.

Changed (backend): all models (company_id), `core/config.py`, `db/session.py`, `api/auth.py`, `api/cases.py`, `api/documents.py`, `api/case_actions.py`, `api/case_reports.py`, `api/signatures.py`, `api/settings.py`, `api/audit.py`, `main.py`, risk scoring / flags / issuer / duplicate / report / workflow / audit / storage / signature / OCR / LLM services, every Celery task, `celery_app.py`, `seed.py`, `start-worker.sh`, `Dockerfile`, `.env.example`, schemas, most tests.

Frontend: added `api/platform.ts`, `hooks/useActingCompany.tsx`, `components/CompanyPicker.tsx`, `components/platform/PlatformShell.tsx`, `pages/PlatformCompaniesPage.tsx`, `PlatformUsagePage.tsx`, `PlatformQueuesPage.tsx`; changed types, routes, nav, settings pages, case queue/detail, dashboard, audit page.

Repo root/docs: `docker-compose.yml` (worker services), `CLAUDE.md`, `README.md`, `docs/multi-tenancy.md`, `docs/processing-queues.md`, `docs/README.md`, this report.